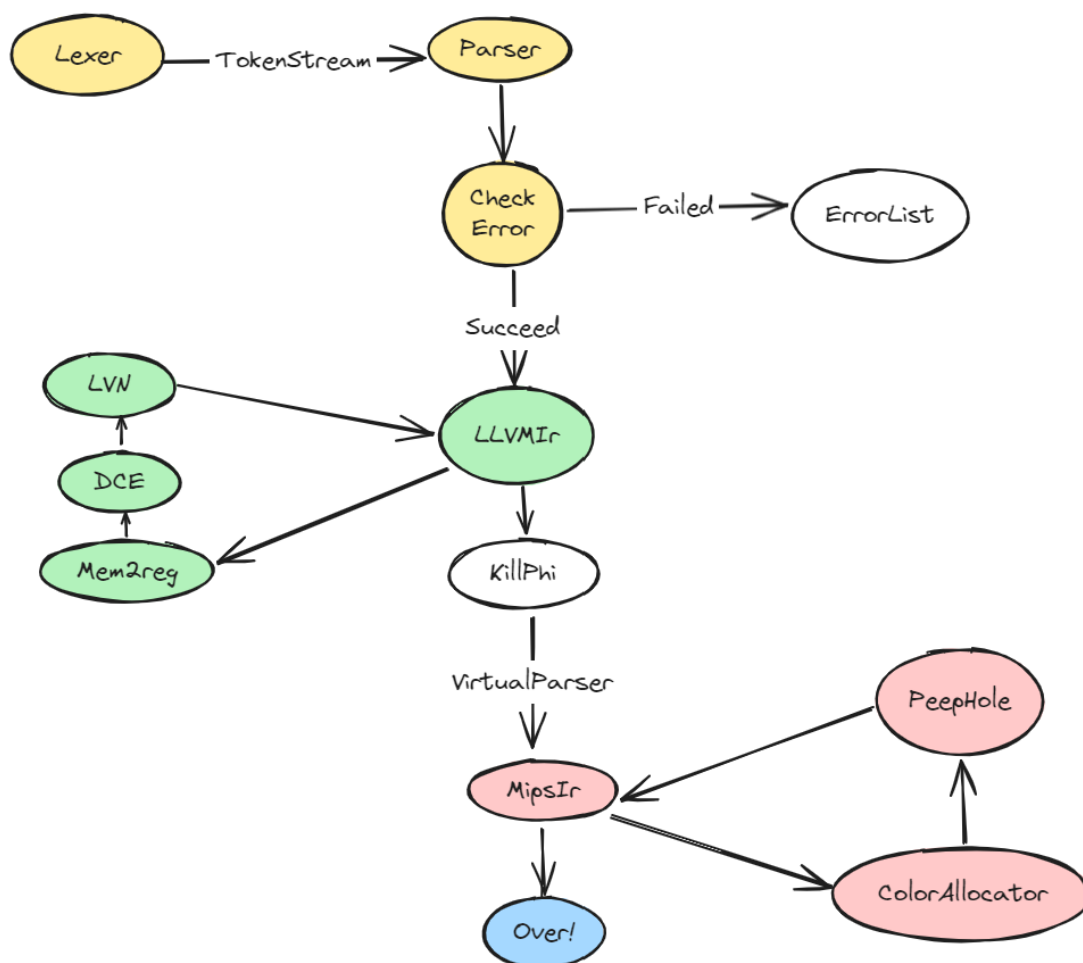


编译器总体介绍

编译器处理流程如下图。



接口设计

除了 testfile.txt 输入 Lexer 的情况，中端的连接通过 LLVM 结构中的 LLVMModule，后端的连接通过，MipsModule 连接，两个部分都有 Parser 进行解析。

文件组织

与大多数编译器相同，分为前端，中端和后端。

前端是语法树，语法节点，和 Lexer，Parser，CheckError 三个功能。

中端包含着 LLVM 的结构，和各种代码优化。

后端包含着 Mips 的结构，MipsParser，MipsAllocator 及优化的功能。

词法分析

这部分在理论考试中很复杂，但是因为 sysY 的文法比较友好，所以仅仅使用普通的状态机即可。

按照要求将组合字符的字符串放在特定的 TOKEN 中即可，整个过程很自然，testfile.txt 扫描一遍即可。

```

void Lexer::tokenize() {
    skipwhitespace();
    // skip single-line comment
    if (peek()=='/' && peek(1)=='/') {
        tokenizeComment();
    }
    // skip multi-lines comment
    else if (peek()=='/' && peek(1)=='*') {
        tokenizeComment_mult();
    } else if (peek()=='\"') {
        tokenizeString();
    } else if (std::isdigit(peek())) {
        tokenizeNumber();
    } else if (std::isalpha(peek()) || peek()=='_') {
        tokenizeword();
    } else {
        tokenizeOperatorOrPunctuation();
    }
}
}

```

编码后的修改

这里对于 '\n' 的处理有一些问题，因为虽然文本中是 \n 但是读入就读入的换行符。

语法分析

这里我修改了文法，因为原先的部分文法比较冗余，我认为只会给之后的自己加大工作量。

比如说，二元运算中的逻辑运算，加减运算等，可以使用一个模板类进行处理，同时将递归文法转变为，节点下包含数组。

如下

```

template<typename T>
class ExpUniform : public ASTNode {
protected:
    T leftOperand;
    vector<TokenType> operators;
    vector<T> operands;
};

```

与此类似的还有一些，没有展开。

我为每个节点都创建了一个类，节点之间指针连接建立抽象语法树，同时，为了方便和形式统一，节点都是抽象类 ASTNode 的继承节点，以便应对评测。

语法分析整体方案采用递归下降，特别之处在于 LVal 和 Exp 的判定区分，这里我是这样处理的。

因为观察到，LVal 是 Exp 的一部分，所以按照 LVal 来解析，如果解析成功，并且满足后面的文法，则视为 LVal

反之视为 Exp。同时需要注意的是需要关闭语法分析的输出，避免评测失败。

编码后的修改

基本没有什么修改，有一点手癖和笔误。

错误处理

我的错误处理和代码生成是分开的。

错误处理和代码生成都需要构建符号表，在实现中我使用了相同的栈式符号表。

```
class Symbol {
    stack<SymbolSetPtr> symbolStack;
    map<string, shared_ptr<stack<VarSymbolPtr>>> varsMap;
    map<string, FuncSymbolPtr> funcsMap;
}
```

原因在于，逻辑清晰便于理解，记录每个 Scope 的变量，和他们对应的一系列有关信息。

这里是信息的储存。

```
class VarSymbol {
    bool isConst;
    bool isGlobal;
    vector<int> lens; // at most 2 dimensions
    vector<int> values; // save it in 1 dimension
    Value *llvmValue;
};

class FuncSymbol {
    bool isVoid;
    vector<int> dims; // stand for the types
    Value *llvmValue;
};
```

错误处理的实现方法是前序遍历语法树，通过记录当前的上下文实现的。

在每个类中实现 checkError 方法，保存例如是否是函数定义，当前变量是否是左值等等。

一个例子如下

```
void FuncFParam::checkError(ErrorCtxPtr ctx, ErrorRetPtr ret) {
    for (const auto &i : expPtrs) {
        i->checkError(ctx, ret);
    }
    vector<int> lens((isArray) ? expPtrs.size() + 1 : 0);

    if (!symbol.insertVar(false, ident.getValue(), lens)) {
        errorList.emplace_back(Exception::REDEFINED_IDENT, ident.getLineNum());
    }
}
```

编码后的修改

基本没有什么修改，有一点手癖和笔误。

代码生成

这部分代码量很多也很复杂，我的中间代码是 llvm。

LLVM

我的 llvm 架构和官方的 LLVM 架构保持一致，几乎所有的变量都继承 Value，有 User 记录使用的 Value，也有 Use 记录每个 Value 的 User。

我认为比较重要的是 Use 这个结构，因为 LLVM 是 SSA 形式的，因此，这个结构就包含了 Def-Use 关系，很多情况下也是维护这个变量即可。

下面是我 LLVM 的结构

```
dir /s /b /a-d
.\BasicBlock.h
.\Function.h
.\GlobalVariable.h
.\Module.h
.\Type.h
.\Use.h
.\User.h
.\Value.h
.\constant\Constant.h
.\constant\ConstantArray.h
.\constant\ConstantInt.h
.\constant\ConstantString.h
.\instruction\AllocaInst.h
.\instruction\AluInst.h
.\instruction\BrInst.h
.\instruction\CallInst.h
.\instruction\GEPInst.h
.\instruction\IcmpInst.h
.\instruction\Instruction.h
.\instruction\LoadInst.h
.\instruction\ParallelCopyInst.h
.\instruction\PhiInst.h
.\instruction\ReturnInst.h
.\instruction\StoreInst.h
.\instruction\ZextInst.h
```

代码生成过程

这个过程与错误处理类似，不同的是我建立了 IrBuilder 类用于封装每个生成的动作，并且保存上下文。

以下是我认为比较重要的节点。

Def 节点

Def 节点的复杂性来自于，有很多的分类标准，是否是常量，是否是数组，是否有初值，是否是全局变量。

而他们相互之间的重复性并不大，因此需要每种情况细致考虑。

短路求值

这个部分我是将所有条件的判断都放在了内容块的前面。

原理如下，将 LOr 和 LAnd 分别判断，通过分析对于当前的 Success 块和 Failure 块来进行抉择。

```
entry:
    br i1 %cmp, label %land.lhs.true, label %lor.lhs.false

land.lhs.true:
    br i1 %cmp1, label %if.then, label %lor.lhs.false

lor.lhs.false:
    br i1 %cmp2, label %land.lhs.true3, label %if.else

land.lhs.true3:
    br i1 %cmp4, label %if.then, label %if.else

if.then:
    br label %if.end

if.else:
    br label %if.end

if.end:
    ret i32 %4
```

伪 SSA 形式

说伪的原因是，这时我通过借用内存实现 SSA。

对于每个变量，在定义时，内存开块空间塞进去，使用时从那个空间拿出来。

接下来会在 mem2reg 优化。

编码后的修改

一开始将 Use 的行为理解错了，Value 中的 UseList 中放的仅仅是他作为 Value 的 Use 而不包括作为 User 的部分，这里在代码优化的时候才发现。

代码优化

mem2reg

这个是放在中端优化的最前边，在做 mem2reg 之前需要先分析数据，生成流图和支配树，具体的生成方法参考的是教程网站上给的算法。

之后就是 mem2reg 的步骤，首先需要构建 def-use，这个的 def use 是根据每条指令生成的，不仅包括 inst 还有 basicBlock。

之后是根据算法插入 phi 指令，简单来说就是根据支配边界将，后边的块插入一个 phi 指令，之后再通过 rename，将他的可能来源替换为前面块的 reachdef，这个 reachdef 的意思就是当前最近的定义，可以使用一个栈来处理。

死代码删除

这个我感觉我做的没问题，但实际用处不大，因为 mem2reg 已经做过一遍死代码删除了，而我又没有做其他的优化，就导致没效果。

死代码删除的原理是，根据每条有用的指令计算他的指令闭包，闭包就是他被谁用了，和用谁了，算完之后得到一个全部的集合，把除了集合之外的指令全删了就是死代码删除。

LVN

没做 gvn 的原因是，做了 gvn 还得做 gcm，做不完了。

lvn 很好写，就是根据每种指令设计一个 hash，然后在后边如果看到相同的 hash 就给替换了，如果是常量，可以直接利用 uselist 把后边指令中的 use 全给替换了。

在 lvn 的过程中可以顺便做一些简单的常量折叠，比如说两个操作数全都是常量的情况下，就可以直接计算出结果，如果是比较的逻辑，例如 $a * 0$ ，就可以直接用 0 来替换。

消除 PHI 指令

我放在了中端优化的最后面，实际上也就是后端的消除 phi。具体的操作来自于 SSA Book，这里记录一下实现的注意事项。

首先要对每个函数做插入 parallel copy，是并行赋值，意思就是在于将，phi 指令提前到当前基本块的前置基本块中，因为 parallel copy 是保证了并行赋值，所以暂时不需要移动。

这里要注意，因为根据 phi 指令的定义，phi 指令的 operand list 中的数量实际上是和他的前置基本块数量是一致，并且顺序也是一致的，因此这里可以直接将 phi 的 operand 放到 parallel copy 中。

```
// because phi's preBb is exactly the same as curBb's preBb
// it is under guarantee by the definition of phi instruction
// so parallel copy inst options' quantity is equal to phi instruction options'
quantity;
    for (auto inst : curBb->instructionList) {
        if (inst->instType != InstType::PHI)
            break;

        for (int i = 0; i < pcList.size(); ++i) {
            pcList[i]->copyPair(inst, inst->operandList[i]);
        }
    }
```

```

    }

}

// kill all the phi Inst;
curBb->instructionList.remove_if([](Instruction *i) { return i->instType
== InstType::PHI; });

```

同时在删除 phi 指令和插入 parallel copy 指令的时候也要记得维护 CFG 以保证之后不会出错。

```

/*
 * what we need to do here: (not necessarily in that order)
 *      ; update cfg info (done in insert bb)
 *      ; change branch instruction
 *      ; insert parallel copy
 */

// last instruction is branch, which is under guarantee by cfg;

```

完成了并行赋值之后，要进行 move 指令的插入，因为硬件上做不到同时赋值。

这里的修改也很简单，如果是既 def 又 use，就先放到一个 tmp 里边，不过会产生很多的 move 指令，这里多就多吧.....

图着色寄存器分配

这部分需要先通过 irParser 生成只有 虚拟寄存器的 Mips 指令流，然后根据后端的指令再次构建 def use，真的麻，在这还得再做一遍。

图着色的完全体是可以，计算预着色，和传送指令的，不过在竞速中，实际根本不会出现那么多的 spillNode，也就是说给你的寄存器绝大多数情况下都用不玩，于是我就没做预备着色和传送指令，将一些 可能会被用上的指令干脆不分配了

最后可以用的寄存器一共有 23 个。

图着色的原理就是根据上边提到的def use 构建冲突图，得到了一个全都是虚拟寄存器的图，这个时候挨个删，先删度小于寄存器数量的点，删不动了就随便拿出来一个当成 spillNode，毕竟不追求性能，之后将所有spillNode 的点用到的地方都变成内存的读取，最后根据用到的寄存器保存上下文 和调整栈的使用。

以下是我规划的栈的使用。

```

/*
 * ----- view of stack -----
 *
 * when entering a function
 *
 *      | # 7th arg
 *      | # 6th arg
 *      | # 5th arg
 *      ----- <-- $sp without fixing
 *      |
 *      | context saving where I'm planning
 *      |

```

```

*      -----
*      |
*      |
*      |  alloca
*      |
*      |
*      |
*      -----
*      $sp <--- here is stack pointer
*
*  $sp is always at the bottom, in order to reduce calculation
*/

```

乘除法优化

我的乘法仅仅做了 当乘数为 2^n 的情况，除法 参考 **Division by Invariant Integers using Multiplication** 论文中的做法，复现算法。

指令优化

这个地方就是 后端 IRParser 的使用要简单一点，非常的琐碎，比如伪指令的选择等。

编译器致谢

- 代码优化参考，毕昇杯作品，喵喵队，<https://gitlab.eduxiji.net/educg-group-18973-1895971/compiler2023-202310006201934>
- 错误处理和代码生成架构参考，<https://github.com/gyp2847399255/SysY-compiler>
- 图着色算法参考，虎书